

Mid Semester Examination (September 2020)

**High Performance Scientific Computing
(COMP3320/COMP6464)**

Writing period: 90 Minutes duration

Include your student number on each document you submit.

Submit the answers to the exam paper through Wattle by 11.30 AM.

This exam is marked out of 100. You must answer all questions.

Your mark for this exam will contribute at most 20% of your total course mark, according to the marking scheme given on the course web page.

Answer all questions yourself and DO NOT COPY FROM OTHER SOURCES!

Contact details:

Zoom Meeting ID: 976 4876 8644

Email: t.huber@anu.edu.au

Phone: 6125 9346

Question 1: Timing [15 total marks]

(a) In computing two measures of time are used to analyse the execution speed of code. Name the two measures of time and briefly discuss advantages and disadvantages of their use.

[5 marks]

(b) Give two fundamentally different scenarios of code execution where the two timing measures identified in Q1a report significantly different results, and explain the reason.

[5 marks]

(c) When is it better to have few powerful computers rather than many less powerful computers? When would the opposite be true?

[5 marks]

Question 2: Floating point arithmetic [20 total marks]

(a) Machine represented floating point number systems are finite and discrete. The number of normalized floating point numbers can be written as:

$$2(\beta-1) \beta^{t-1} (U-L+1) + 1$$

Where β is the base, t is the precision, U is the maximum value of the exponent, and L is the minimum value of the exponent. For $\beta=2$, $t=3$, $U=2$, $L=-2$ there are 41 numbers. For this floating point system:

- (i) Write the value of the three largest positive numbers.
- (ii) Write the value of the three smallest (non-zero) positive numbers.
- (iii) What are subnormal numbers, and where would they go?

[6 marks]

(b) The following code illustrates the effects of both approximation and rounding errors:

```
#include <stdio.h>
#include <math.h>
int main(void)
{ float h,x;
  int i;
  x=12.3456789;
  for (i=0, h=0.1; i<13; i++, h*=0.1)
    printf("%16.14f %8.1e %16.14f \n", 2.0*x, h,
      (((x+h)*(x+h))-(x*x))/h);
  return 0;
}
```

It computes the derivative x^2 both analytically and using a finite difference method:

$$\frac{dx^2}{dx} = 2x \approx \frac{(x+h)^2 - x^2}{h}$$

Where h is the step size used in the finite difference. The code uses $x = 12.3456789$ and various step sizes. It gives the following output:

```
24.69135780000000 1.0e-01 24.791357800000007
24.69135780000000 1.0e-02 24.701357800000077
24.69135780000000 1.0e-03 24.692357800000644
24.69135780000000 1.0e-04 24.69145779997461
24.69135780000000 1.0e-05 24.69136780121061
24.69135780000000 1.0e-06 24.69135878868655
24.69135780000000 1.0e-07 24.69135779392672
24.69135780000000 1.0e-08 24.69136006766347
24.69135780000000 1.0e-09 24.69136006766347
24.69135780000000 1.0e-10 24.69136006766347
24.69135780000000 1.0e-11 24.68993898219195
```

24.69135780000000 1.0e-12 24.69846549502107
24.69135780000000 1.0e-13 24.72688720445147
,

- (i) Explain how this program illustrates both rounding error and approximation error.
- (ii) For any given value of h , state how you would modify the code to: (A) Decrease the effect of rounding errors; (B) decrease the effect of approximation errors.
- (iii) For $h=10^{-7}$ write the expression for the relative error of the numerical derivative. [Note: You do not need to evaluate the expression].

[14 marks]

Question 3: CPU Architecture [25 total marks]

(a) Modern processors make extensive use of pipelining. Name three concepts other than pipelining that have been developed to improve processor performance and briefly describe how they work.

[6 marks]

(b) Data dependencies impact on the ability to pipeline. Give a few lines of code containing a for loop where there is a data dependency between iterations of the loop. Clearly indicate the dependency.

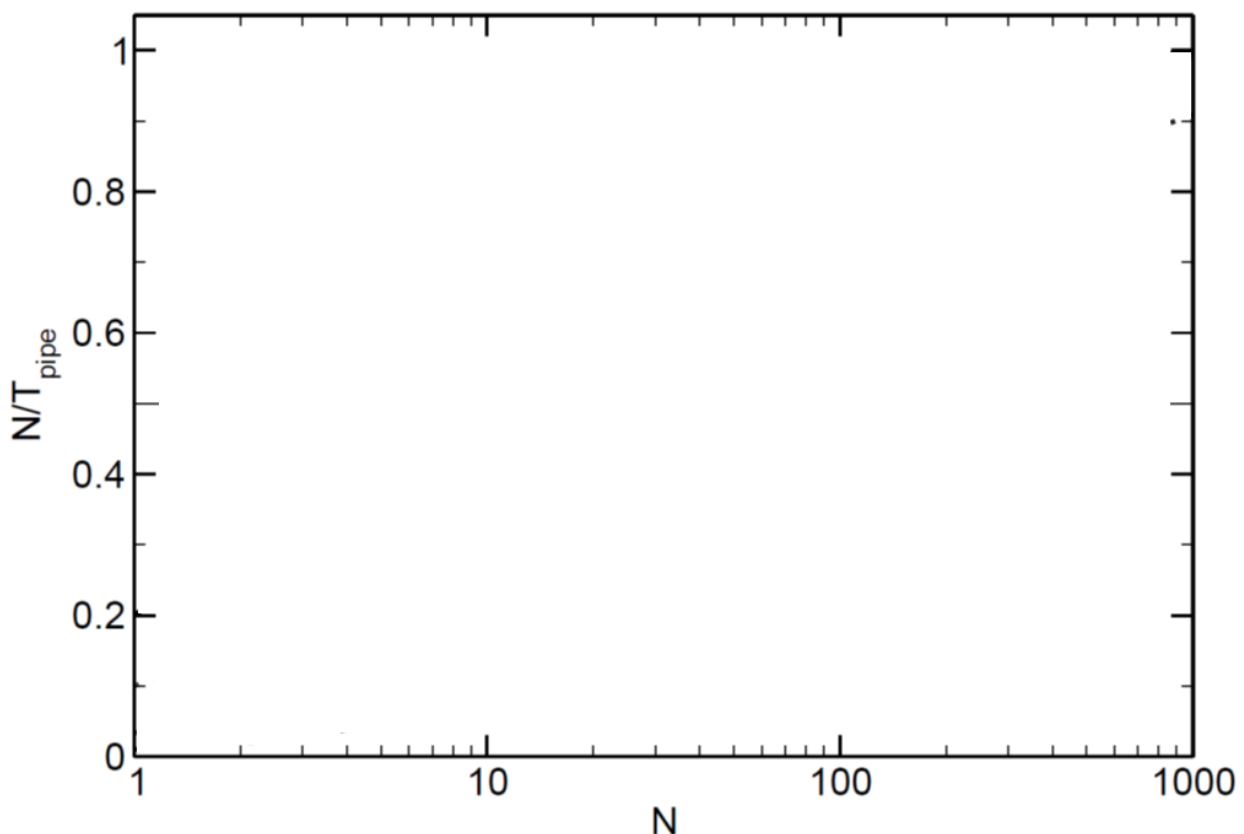
[5 marks]

(c). Code branches impact on pipeline performance. Briefly explain two different ways that modern hardware and compilers may handle branches in loops to improve performance.

[6 marks]

(d) Sketch (and label) in the following graph the pipeline throughput N/T_{pipe} as a function of N independent operations for:

- (i) a small size pipeline depth
- (ii) a medium size pipeline depth
- (iii) a large size pipeline depth



[8 marks]

Question 4: Memory architecture [15 total marks]

(a) Sketch a C code snippet that lets you determine the memory bandwidth of a computer and to estimate the caches at various levels of the hierarchy. Sketch (i.e. graph) the expected bandwidth of your code on a recent computer architecture and use this experiment to estimate the bandwidth and L1 cache size. Justify your answer.

[9 marks]

(b) Explain spatial and temporal locality of reference, and how modern computer architectures take advantage of both.

[6 marks]

Question 5: Performance tuning [25 total marks]

(a) You are running on a machine with:

- Load/store latency of 3 cycles to L1 cache
- Floating point add or multiply latency of 3 cycles
- Superscalar architecture capable of 1 load/store, 2 floating point operations (but no fma instruction) and 2 integer operations per clock cycle.

Consider the following loop in which all variables are independent. You can assume that there is no aliasing and no loop overhead (i.e. branching and increment of i are for free):

for ($i=0$; $i<2*N$; $i++$)

$y[i] = y[i] + a * x[i]$

- (i) What is the minimum number of clock cycles that it will take to execute one loop iteration, when the compiler has not performed loop unrolling and pipelining? Give an instruction schedule to explain your answer.
- (ii) What is the speedup achieved when the loop is unrolled 2 times? Include the new code and a new instruction schedule in your answer.

[10 marks]

(b) The following C code snippet performs a vector-matrix multiply. The variables a, b, and c are single precision floating point variables, and you can assume that variables are not aliased.

```
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        a[j][i] = b[j][i] * c[i]
    }
}
```

(i) Rewrite the C code to improve serial performance on an Intel i7 processor without changing code functionality.

[5 marks]

(c) Shown below are three functionally identical C functions to perform a matrix-matrix multiplication. For these functions also shown below are the anonymized benchmark results obtained on an Intel Sandybridge system. Assign which benchmark result (A, B or C) corresponds to which C function, and briefly explain the performance differences.

```
void matmuldot(int n, double a[], double b[], double c[])
{
    int i,j,k;
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            for (k=0; k<n; k++)
                c[i*n+j]=c[i*n+j]+a[i*n+k]*b[j*n+k];
}
void matmuldax(int n, double a[], double b[], double c[])
{
    int i,j,k;
    for (i=0; i<n; i++)
        for (k=0; k<n; k++)
            for (j=0; j<n; j++)
                c[i*n+j]=c[i*n+j]+a[i*n+k]*b[k*n+j];
}
void matmulout(int n, double a[], double b[], double c[])
{
    int i,j,k;
    for (k=0; k<n; k++)
        for (i=0; i<n; i++)
            for (j=0; j<n; j++)
                c[i*n+j]=c[i*n+j]+a[i*n+k]*b[j*n+k];
}
```


Code	NSIZE	Time(sec)	MFLOPS
A	100	0.001110	1571.752319
B	100	0.000927	1882.578735
C	100	0.000946	1872.674683
A	1000	1.084468	1886.433838
B	1000	0.912717	2003.138062
C	1000	8.592217	1523.240845
A	2000	9.898019	1629.940552
B	2000	8.761745	1748.156616
C	2000	86.124147	1310.452026

[10 marks]